

Why Are Zeros Good And Why So Many of Them?

Edited by Hermawan Nugroho
Source by Phil Sewell

Why so many zeros?

We are going to consider

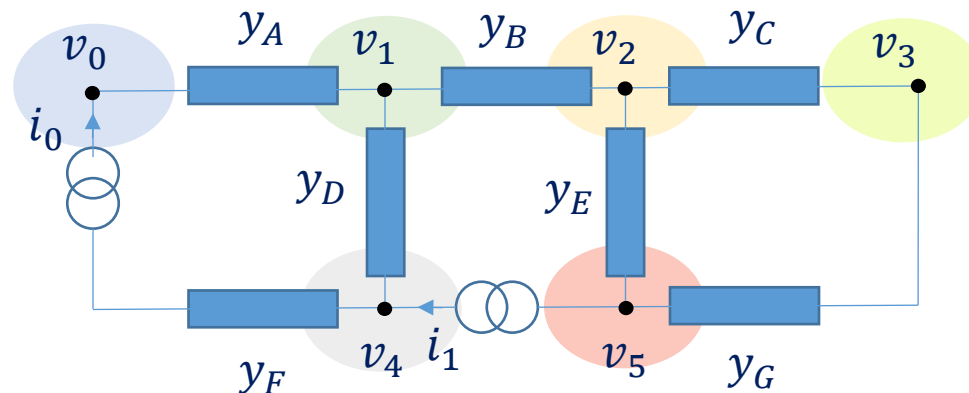
- How matrices which are dominated by zero element values can be managed so efficiently compared to general matrices
- Why so many problems of engineering important lead to such matrices
- It is true to say that modern CAD dealing with large scale problems only functions because of the techniques we will discuss
- A very brief view of computer parallelisation which is a key enabler

As usual, let us convince ourselves using one of our shared understanding examples that this is a topic worthy of study...

Why so many zeros?

Example: Nodal Analysis leads to matrices of the form

$$\begin{pmatrix} 0 \\ 0 \\ -i_0 \\ 0 \\ -i_1 \\ i_1 \end{pmatrix} = \begin{pmatrix} (y_A) & -y_A & 0 & 0 & 0 & 0 \\ -y_A & (y_A + y_B + y_D) & -y_B & 0 & -y_D & 0 \\ 0 & -y_B & (y_B + y_C + y_E) & -y_C & 0 & -y_E \\ 0 & 0 & -y_C & (y_C + y_G) & 0 & -y_G \\ 0 & -y_D & 0 & 0 & (y_D) & 0 \\ 0 & 0 & -y_E & -y_G & 0 & (y_E + y_G) \end{pmatrix} \begin{pmatrix} v_0 \\ v_1 \\ v_2 \\ v_3 \\ v_4 \\ v_5 \end{pmatrix}$$



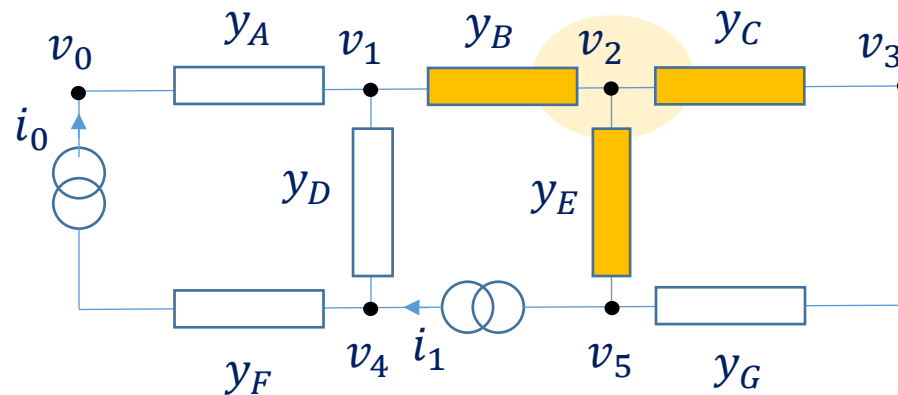
We see a fair few zero entries

Physically describe the situation when there is a zero entry?

Why so many zeros?

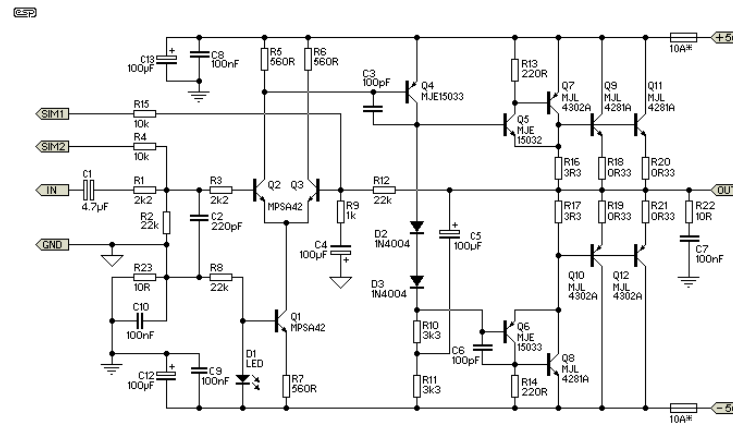
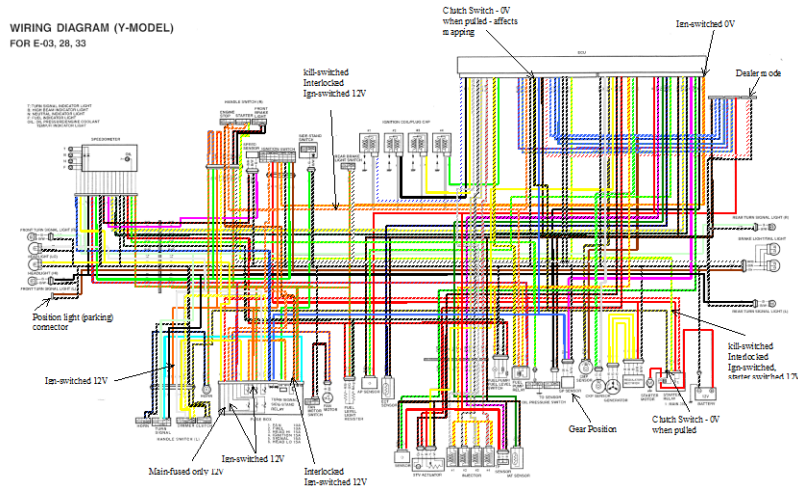
Non zero elements correspond to connections between nodes

$$\begin{pmatrix} 0 \\ 0 \\ -i_0 \\ 0 \\ -i_1 \\ i_1 \end{pmatrix} = \begin{pmatrix} (y_A) & -y_A & 0 & 0 & 0 & 0 \\ -y_A & (y_A + y_B + y_D) & -y_B & 0 & -y_D & 0 \\ 0 & -y_B & (y_B + y_C + y_E) & -y_C & 0 & -y_E \\ 0 & 0 & -y_C & (y_C + y_G) & 0 & -y_G \\ 0 & -y_D & 0 & 0 & (y_D) & 0 \\ 0 & 0 & -y_E & -y_G & 0 & (y_E + y_G) \end{pmatrix} \begin{pmatrix} v_0 \\ v_1 \\ v_2 \\ v_3 \\ v_4 \\ v_5 \end{pmatrix}$$



Why so many zeros?

What proportion of elements are likely to be zero? Elements are **connections**



Many real world examples where the *connectivity* is from each *node* to just to a few others

Why so many zeros?

What proportion of elements are likely to be zero?



Wave (and diffusion) phenomena where each "point" responds only after its neighbours change lead to "next neighbour only" connectivity matrices

In a square N by N matrix with N^2 elements only $\sim 7N$ of them are non-zero

E.g. 70000 in 100000000 i.e. $<1\%$

Why so many zeros?

*Matrices with relatively few non-zero elements are called **sparse***

*Matrices without many zero elements are called **dense***

If we apply all our previous manipulation techniques, to sparse matrices, e.g.

- *matrix multiplication,*
- *QR reduction using Given's and Householder for solving*
- *LU decomposition for solving*

There is no general advantage at all because of all the zeros!

Is this a surprise?

Why so many zeros?

Example:

Matrix multiplication using sparse matrices

$$\begin{pmatrix} A_{00} & & A_{03} \\ & A_{12} & \\ & & A_{23} \\ & & & A_{33} \end{pmatrix} \begin{pmatrix} B_{01} & & B_{13} \\ B_{20} & B_{21} & \\ & & B_{33} \end{pmatrix}$$

Generally matrix multiplication of a pair of 4*4 matrices requires 64 multiply and adds, but surely here don't need to bother with the "multiply by 0" operations?

Solution: To take advantage of the zeros in this way requires lines in a code of the form

if (element not zero) do multiply and add operation

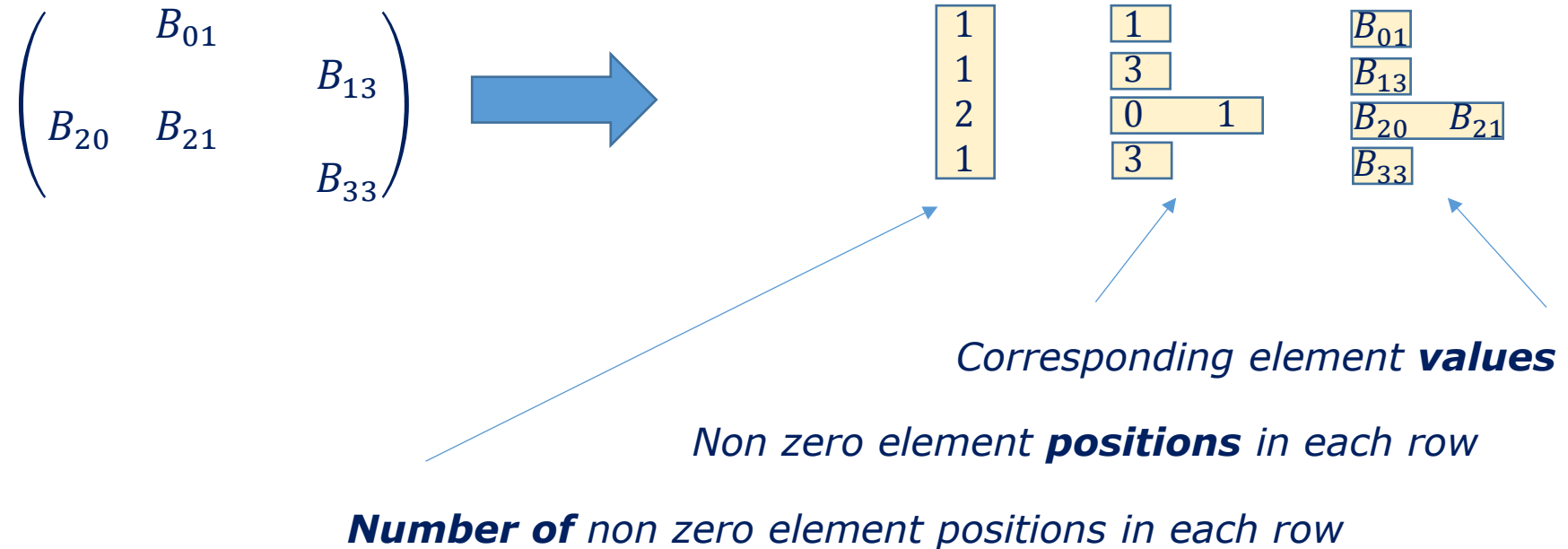
Any time gained by not doing multiply and adds is more than outweighed by the time taken to do all the tests

Sparse Matrix Storage

Sparse matrices are not efficiently stored in a computer as 2D arrays

Much better alternatives such as *Compressed Sparse Row (CSR) Storage*

Example:



Quasi-optimal memory use and easy to code so that multiplies by 0 don't occur -

NOW we get the substantive speed up with sparse matrix multiplication

Sparse matrix MULTIPLICATION can be made very computationally efficient

BUT!!!!!!!

SOLVING sparse matrix problems by the usual techniques does not just involve sparse matrices

Example: Solve $Ax = b$ for x using QR decomposition (or similar)

Solution:

Very sparse

Generally dense


$$A = QR$$

$R = Q^T b$ A **dense multiply** followed by a **dense upper triangular solve**

*Even using CSR, **sparse** matrix solving with approaches used for general matrices involves **dense** operations*

Sparse Matrix Storage

So is this such a problem? Put realistic numbers in for the matrix size

100 million unknowns; elements stored as double precision (16 bytes)

Think of it as a nodal analysis with each element connected to 7 others

Dense matrix has 10^{16} elements (10 peta, i.e. 10,000 Tera elements) requiring
160 Peta Bytes of RAM just to store the matrix, let alone attempt to solve it

Sparse matrix has 700 million elements; CSR needs 100 million integers, 700 million position integers and 700 million element double values requiring
13.4 Giga Bytes of RAM to store the matrix

It's not a question of whether we would PREFER the efficiency of sparse matrices, we really have NO CHOICE on real computers

Why Are Zeros Good And Why So Many of Them?

- Multiplies by 0 can be avoided if we use for example CSR storage
- Many engineering problems are naturally low order connectivity problems
- Sparse "input" does not mean "all sparse processing"

We need a radically new approach to solving sparse matrix problems

**Can we take advantage of what is easy
(for large sparse problems),**

➤ matrix multiplication

**to do what is hard?
(for large sparse problems),**

➤ matrix solving

If you cannot "solve" a problem

Guess the answer

See how well it "works as a solution"

Adjust your guess

See how well it "works as a solution"

Adjust your guess

See how well it "works as a solution"

Adjust your guess

See how well it "works as a solution"

...

And hope things settle down to an ever closer estimate of the true solution

*Such repeated adjust and try approaches are referred to as **iterative approaches***

Four key aspects:

Guess the answer: You might think this is the hard bit, but usually, a *random set of values* is OK

See how well it "works as a solution": Easy and cheap to do - see next example

Adjust your guess: How? This is actually the hard bit and different approaches here are what distinguish the different methods used

And hope things settle down to an ever closer estimate of the true solution; not exactly sound engineering, rather we seek some assurances from a mathematician that our "adjustment strategy" is bound to lead to a "converged" answer - see example in a moment

Example: Given an estimated solution, $\tilde{x}$, to the problem $Ax = b$, identify a way of quantifying how close the estimate is to the true solution

Solution: In words, Ax is supposed to multiply out to b , so if it doesn't what is the difference between Ax and b ?

$$e = b - A\tilde{x} \quad \text{"e for error"}$$

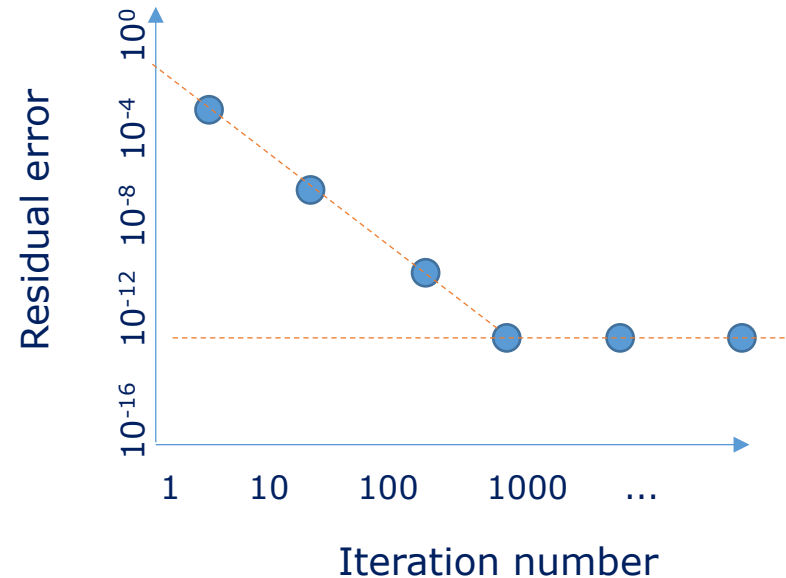
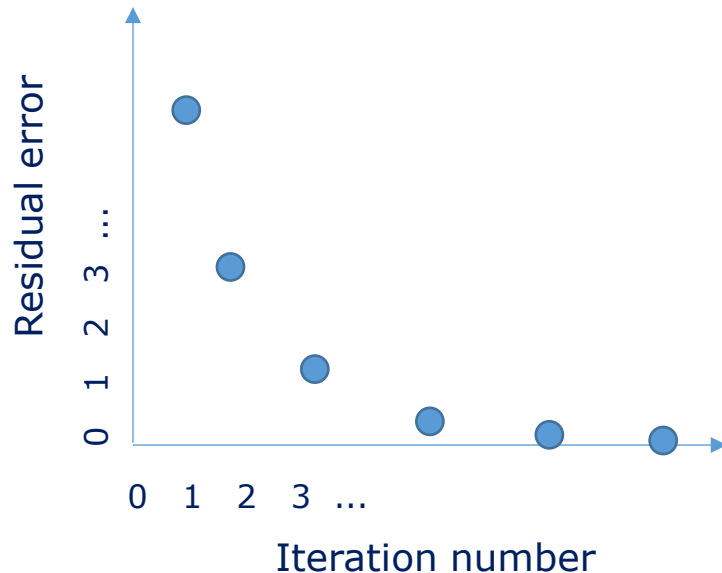
The error is a *column matrix* and we want a *simple error number*

A useful measure of the error is $e^T e$, i.e. the norm of e as this can only be 0 if *all* the individual elements of e are 0

Note: In most text books etc, they use an r not e to denote error - it stands for the *residual error*. I will also use this from now on

Example of Convergence: If we produce a sequence of (hopefully) better estimates of the solution to $Ax = b$, labelled $x_0, x_1, x_2 \dots, x_{N-1}, x_N$ then *convergence* means that $x_{N-1} \approx x_N$ if N is large enough

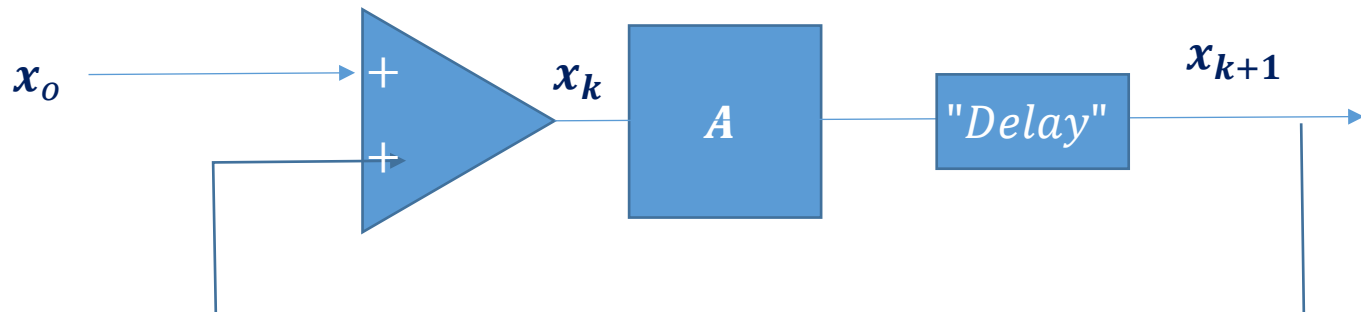
Using our previous error measure,



The 2nd version is the more professionally valid; We often see a nice downward trend on log plots and levelling off at computer *machine precision*

Experiment on the Adjustment Strategy: Produce a sequence of vectors labelled $x_0, x_1, x_2, \dots, x_{N-1}, x_N$, such that $x_k = Ax_{k-1}$ where A is square, real, and symmetric

In words this like a perfect *positive* feedback loop



Note well that the "system" response is obtained by a matrix multiply, not a solve

Experiment on the Adjustment Strategy: Produce a sequence of vectors labelled $x_0, x_1, x_2, \dots, x_{N-1}, x_N$, such that $x_k = Ax_{k-1}$ where A is square, real, and symmetric

Let the initial guess be x_0

We know that the matrix A has eigensolutions $\{Y, k\}$ so that any vector, i.e. x_0 can be represented in the form

$x_0 = Yc$ where c is a column of coefficients and Y the matrix of eigenvectors

$$x_1 = Ax_0 = AYc = Ykc$$

$$x_2 = Ax_1 = AYkc = Yk^2c$$

...

$$x_N = Yk^Nc$$

$$\mathbf{x}_o = \mathbf{Y} \mathbf{c}$$

$$\mathbf{x}_N = \mathbf{Y} \mathbf{k}^N \mathbf{c}$$

Let's expand this to clarify the effect

$$\mathbf{x}_o = (\mathbf{Y}_o \quad \mathbf{Y}_1 \quad \mathbf{Y}_2 \quad \dots) \mathbf{c}$$

$$\mathbf{x}_N = (\mathbf{Y}_o \quad \mathbf{Y}_1 \quad \mathbf{Y}_2 \quad \dots) \begin{pmatrix} k_0^N & & & \\ & k_1^N & & \\ & & k_2^N & \\ & & & \dots \end{pmatrix} \begin{pmatrix} c_0 \\ c_1 \\ c_2 \\ \dots \end{pmatrix}$$

If k_0 is bigger than all other eigenvalues, then

k_0^2 is much bigger than all other k_i^2 and

k_0^3 is much much bigger than all other k_i^3 and

k_0^4 is much much much bigger than all other k_i^4 and by the time we get to $N=100$,

$$\mathbf{x}_N = \mathbf{Y}_o k_0^N c_0$$

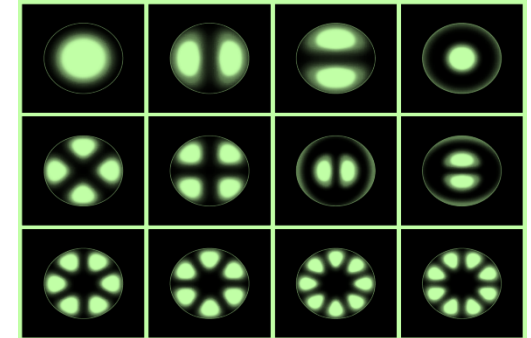
The "shape" of the eventual data produced *is purely the dominant eigenvector*

Repeated iteration of a simple algorithm that converges - Specifically useful?

Iterative Approaches

$x_N = Y_o k_0^N c_0$: The "shape" of the eventual data produced is *purely the dominant eigenvector*

Random light pattern into our fibre model is mix of eigenvectors



Repeated "propagation along fibre elements"
(just the model we looked at)

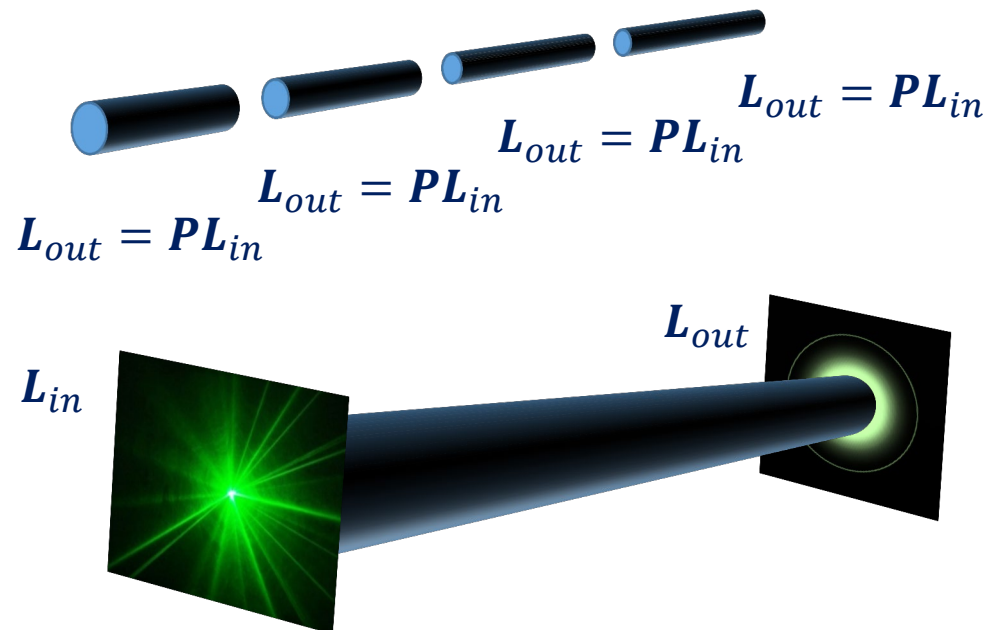
Reveals the dominant eigenvector using only

SPARSE MATRIX MULTIPLIES

Simple adjustment strategy!

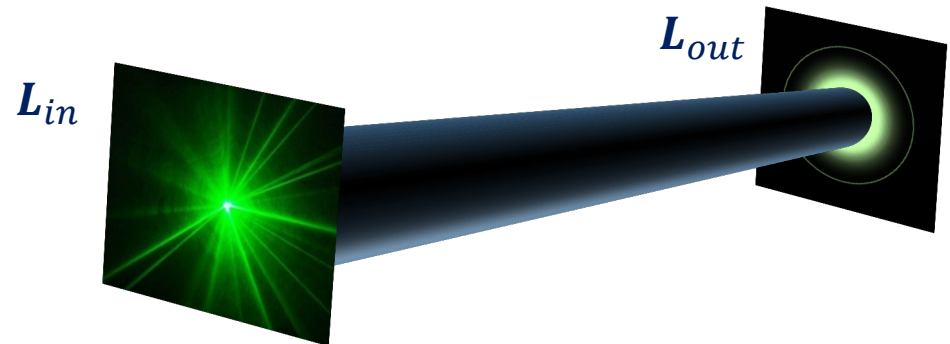
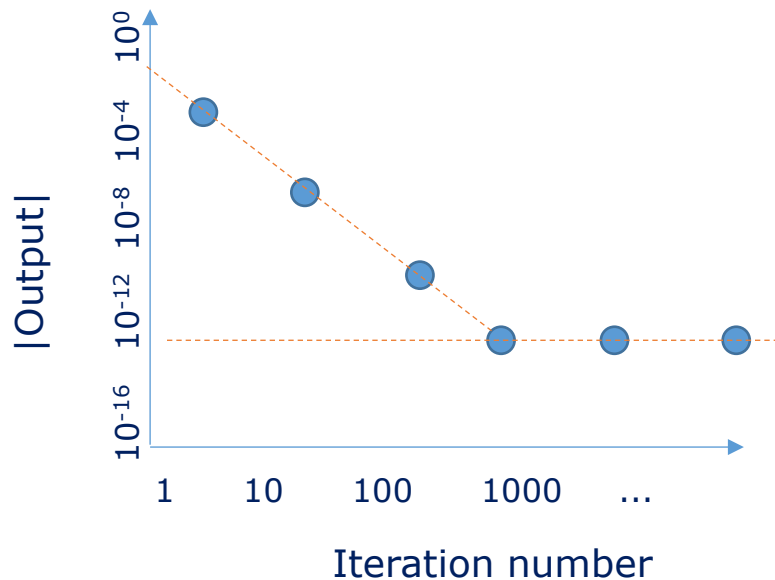
This class of methods are called

Power methods. Search web on *Shifted Inverse Power* method for practical scheme



NOTE VERY WELL: This is a typical use of eigensolutions as a means of undertaking an *analysis* of a numerical algorithm

Stability: The Magnitude of the output scales as k_0^N and we can use the *asymptotic* behaviour to evaluate the eigenvalue as well as obtain the eigenvector



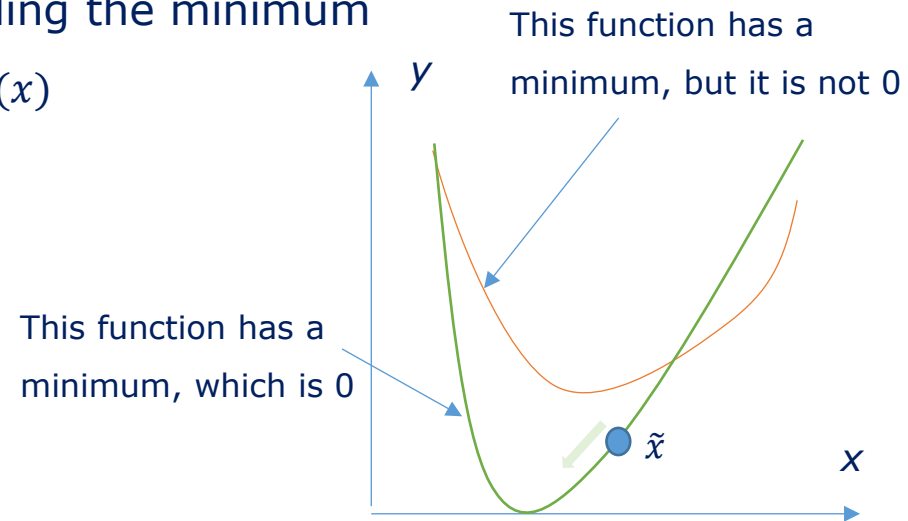
Example: Consider viewing our adjustment strategy as "minimising a function"

$\mathbf{r} = \mathbf{b} - \mathbf{A}\tilde{\mathbf{x}}$ Scalar measure of error is $\mathbf{r}^T \mathbf{r} = \|\mathbf{r}\|^2$

Solution: Compare this to the task of finding the minimum of a simple function of a variable x , $y = f(x)$

How to find a minimum

Solve "gradient = 0"?



This doesn't help because finding where $\frac{d\|\mathbf{r}\|}{dx} = 0$

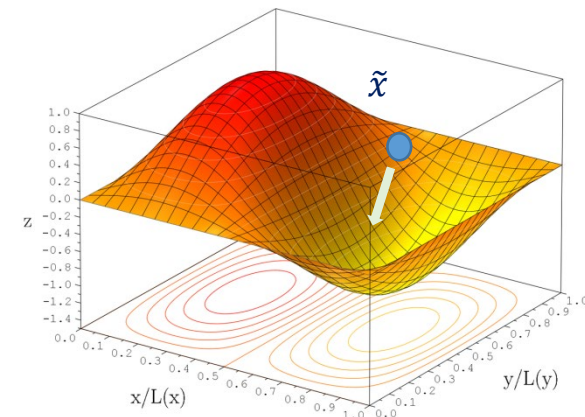
requires us to solve $\mathbf{A}\mathbf{x} = \mathbf{b}$ which is what we are trying to do in the first place!

However, what we can learn from the picture is, given an estimated solution $\tilde{\mathbf{x}}$, we must head in a downhill direction to reach the exact solution

Iterative Approaches

We can learn from the picture that, given an estimated solution $\tilde{x}$, we must head in a downhill direction

Noting that in the matrix case it is an N dimensional plot. For example, in 2D, so we head in the *steepest downhill direction*

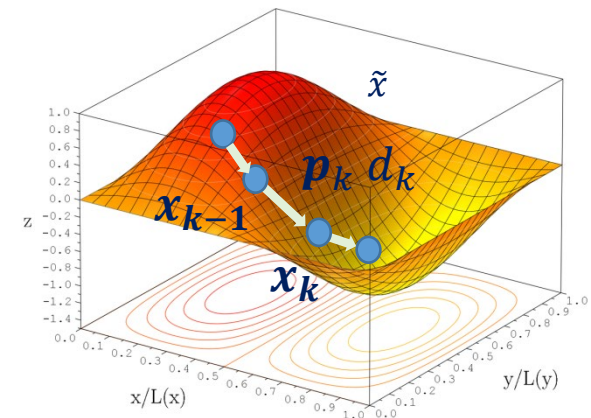


Example: As we iterate through the sequence of estimated solution vectors labelled $x_0, x_1, x_2 \dots, x_{N-1}, x_N$

We need a rule for getting such that x_k from x_{k-1}

We now chose this to express this a direction in which to make the change and a length of change

$$x_k = x_{k-1} + p_k d_k$$



The University of Nottingham

25

Can we take advantage of what is easy to do what is hard?

- Yes if we adopt iterative approaches - relying on efficient matrix multiplies
- Steepest descent illustrates this

Some more thoughts:

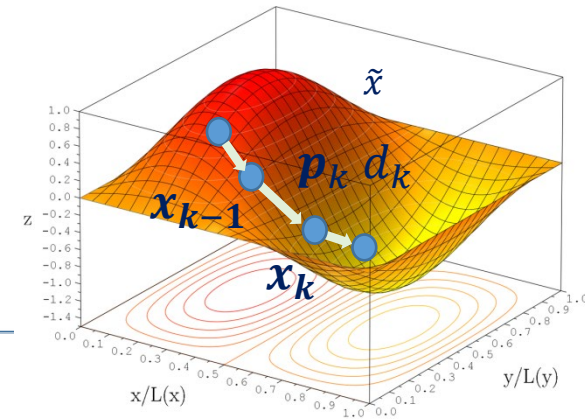
- How many flavours of iterative methods are there?
- How long do they actually take to converge?
- Parallel computing - is that relevant here?

Show me another example of an iterative method

The Minimal Residual Algorithm

The Minimal Residual Algorithm:

for solving $Ax = b$



Start estimate x_0

Do until convergence

$$r_k = b - Ax_k$$

Current error column - will be the direction of the change, p_k

$$d_k = \frac{r_k^T A r_k}{r_k^T A^T A r_k}$$

The length scaling of the step

$$x_{k+1} = x_k + d_k r_k$$

Improved solution estimate

Enddo

It can be shown that: The above algorithm is guaranteed to converge if A is

~~symmetric and~~ positive definite

Deals with a broader range of matrices

Is this going to take long - just how many algorithms do you expect the poor students to memorise?

- This subject might require about 60 credits to *start* to do properly and such a module would generally be called a module in *Linear Algebra*
- I don't expect you remember or use any of them in detail for the exam. I do ask you to be able to generally discuss the general issues influencing the choices a real engineer makes - see the self study sheet for explicit examples to clarify this
- So let's at least have a quick look at the real breakthrough variant

*Hestenes and Stiefel "Methods of **Conjugate Gradients** for Solving Linear Systems". Journal of Research of the National Bureau of Standards, 1952*

The Conjugate Gradient Algorithm

Start estimate x_0 and $p_0 = r_0 = b - Ax_0$

Do until convergence

$$\alpha_k = \frac{r_k^T r_k}{p_k^T A p_k}$$

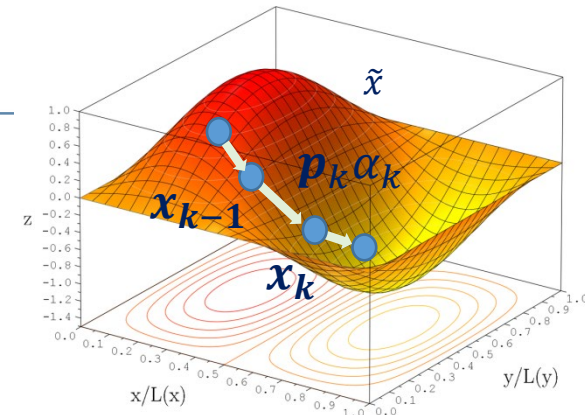
$$x_{k+1} = x_k + \alpha_k r_k$$

$$r_{k+1} = r_k - \alpha_k A p_k$$

$$\beta_k = \frac{r_{k+1}^T r_{k+1}}{r_k^T r_k}$$

$$p_{k+1} = r_{k+1} + \beta_k p_k$$

Enddo



Our concern is not to prove this, but I just want highlight to you the key features

- The pattern is that we maintain 3 column vectors, x , r , p and each of these has an iterative update expression determined from the previous values
- Only matrix multiplications - highly efficient for sparse matrices

What about convergence - guaranteed? Slow, or fast? Critical!

It can be shown that: For a symmetric matrix, $A^{(N \times N)}$, the CG algorithm is guaranteed to converge to the exact answer in no more than N iterations

Except that in practice this never even vaguely happens for a real problem - *sorry!*

The wonders of mathematical *theory* versus engineering *reality* - *welcome to my world*

The root cause of this failure is the chronic accumulation of numerical rounding errors in multiplying out all those large (albeit sparse) matrices

However there are one class of problems which are virtually ideal and this gives an opening to broaden this to all matrices

It is observed that: The CG algorithm is virtually ideal for matrices that are close to diagonal in nature

$$\begin{pmatrix} 5 & 10^{-2} & 10^{-4} & 10^{-6} \\ 10^{-2} & 1 & 10^{-2} & 10^{-4} \\ 10^{-4} & 10^{-2} & 2 & 10^{-2} \\ 10^{-6} & 10^{-4} & 10^{-2} & 1 \end{pmatrix}$$

Such a matrix is called ***diagonally dominant***

Might only need 1 or 2 CG iterations for convergence to machine precision

Chance of your problem being *diagonally dominant* - zero!

However ...

Example: Solve $Ax = b$ for x if A is a large space matrix using *preconditioned* CG

Solution: We seek a matrix Z that

- Is sparse
- An approximation to A^{-1}

$$ZAx = Zb$$

ZA is intended to be close to I , e.g.

$$\begin{pmatrix} 1.0001 & 10^{-2} & 10^{-4} & 10^{-6} \\ 10^{-2} & 0.9999 & 10^{-2} & 10^{-4} \\ 10^{-4} & 10^{-2} & 1.001 & 10^{-2} \\ 10^{-6} & 10^{-4} & 10^{-2} & 1.0000 \end{pmatrix}$$

So we apply the Conjugate Gradient algorithm to $ZAx = Zb$

How to find the *preconditioner matrix*, Z ?

Well that's part of those other 55 credits of module we just don't have!

Example: One approach to solving sparse or dense matrices, $Ax = b$, is to split the matrix into an easy and a hard part and iteratively perform an *easy solve*

Split $A = D + E$ where D are the diagonal elements of A and E contains the remaining components. For example,

$$A = \begin{pmatrix} A_{00} & A_{01} & A_{02} \\ A_{10} & A_{11} & A_{12} \\ A_{20} & A_{21} & A_{22} \end{pmatrix} \quad D = \begin{pmatrix} A_{00} & & \\ & A_{11} & \\ & & A_{22} \end{pmatrix} \quad E = \begin{pmatrix} & A_{01} & A_{02} \\ A_{10} & & A_{12} \\ A_{20} & A_{21} & \end{pmatrix}$$

Gauss Jacobi Iterations: $Ax = Dx + Ex = b$

Start estimate x_0

Do until convergence

Solve $Dx_{k+1} = b - Ex_k$ **Easy diagonal solve**

Enddo

The more practical scheme is Gauss-Seidel - search the web

Can be used as a preconditioner

Some more thoughts:

- How many flavours of iterative methods are there?

Many to deal with a whole range of issues. A strong interaction with real computing issues. Various trade offs between, the class of matrices for which they work well, memory, run time and convergence rate and guarantees

- How long do they actually take to converge?

My experience is that even for very large problems, unless you have strong convergence in less than 100 iterations, it is unlikely to ever converge: *Preconditioning is the absolutely critical must*

- Parallel computing - is that relevant here?

Can we take advantage of the Pattern of data in particular sparse matrices?

- The more we know about the inherent nature of the engineering producing the matrices, the more we can optimise solutions

Example: What patterns do we observe in a matrix corresponding to 1D Diffusion Phenomena?

Solution: We could note how we convert PDEs into matrix problems. An example is thermal modelling which is a Diffusion Phenomenon

It can be shown that: The distribution of temperature and the associated heat flow in a 1D object is analogous to an RC *ladder* circuit

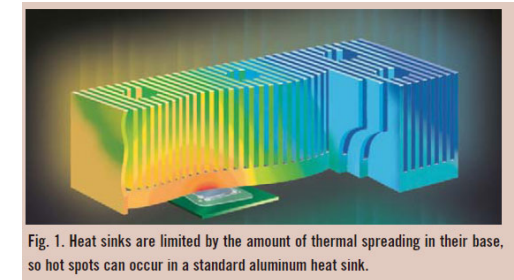
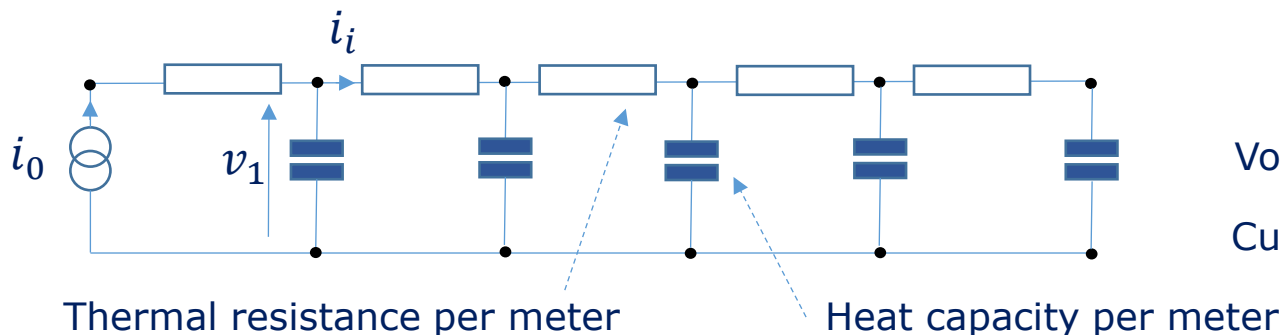


Fig. 1. Heat sinks are limited by the amount of thermal spreading in their base, so hot spots can occur in a standard aluminum heat sink.

Heat Flow:

<http://powerelectronics.com/thermal-management/one-powerful-decade-update-vapor-chamber-heat-sinks>

Voltage $\rightarrow$ Temperature

Current $\rightarrow$ Heat energy flow

What pattern does the nodal analysis matrix to solve this circuit give?

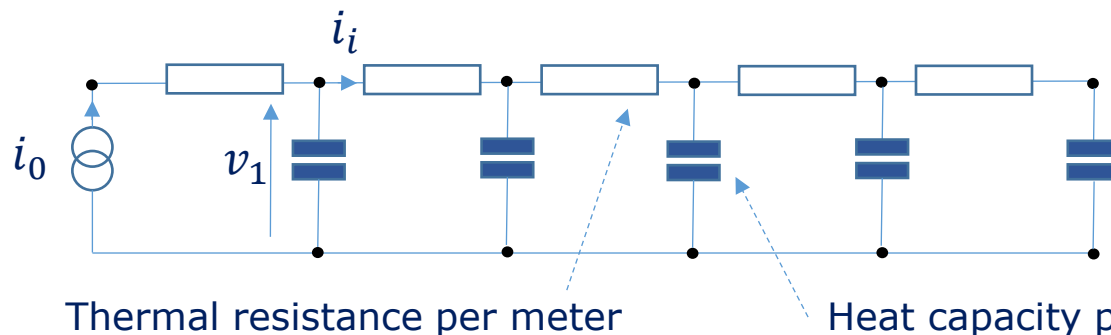
We have simple next neighbour coupling

$$\begin{pmatrix} * \\ * \\ * \\ * \\ * \\ * \end{pmatrix} = \begin{pmatrix} * & * & & & & \\ * & * & * & & & \\ & * & * & * & & \\ & & * & * & * & \\ & & & * & * & * \\ & & & & * & * \\ & & & & & * \end{pmatrix} \begin{pmatrix} v_0 \\ v_1 \\ v_2 \\ v_3 \\ v_4 \\ \dots \end{pmatrix}$$

This is referred to as a **Tridiagonal Matrix**

The reason that this is important is that many of the dense matrix approaches (e.g. LU decomposition and eigen solving, specialise to

efficient forms without needing to resort to general iterative methods



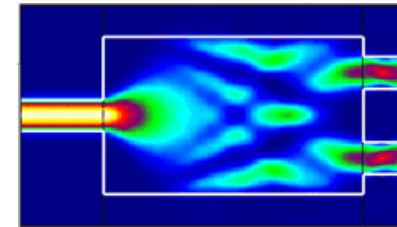
Voltage → Temperature

Current → Heat energy flow

Example: What patterns do we observe in a matrix corresponding to 1D
Wave Phenomena?

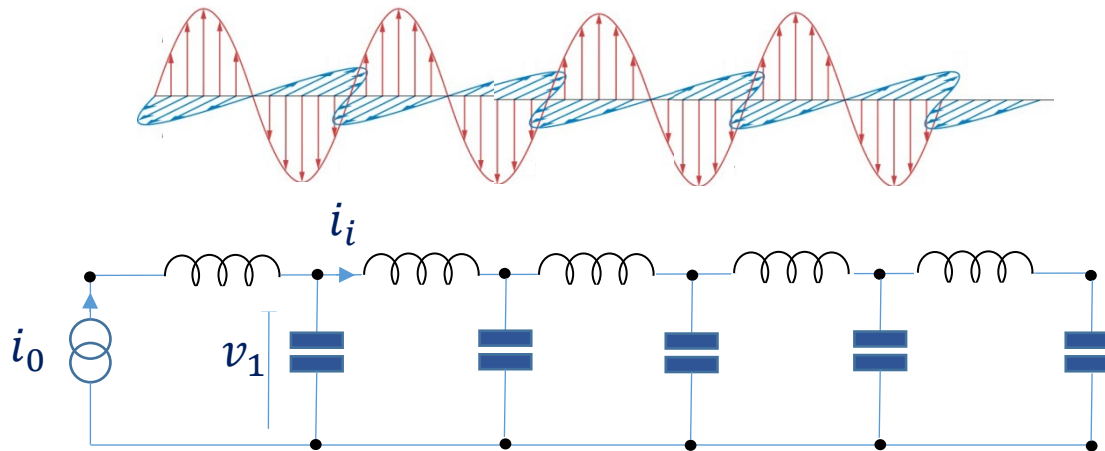
Solution: We could note how we convert PDEs into matrix problems. An example is electromagnetic/light modelling which is a **Wave Phenomenon**

It can be shown that: The distribution of light/fields/RF signals on cables/PCB tracks ... in a 1D object is analogous to an LC *ladder* circuit



Photonics:

<https://www.photond.com/products/fimmprop.htm>

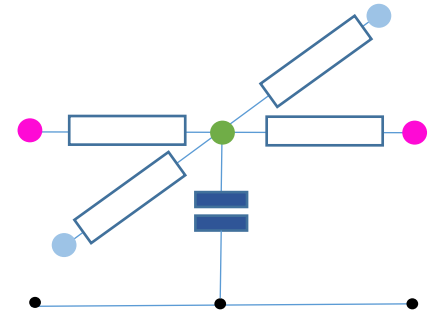
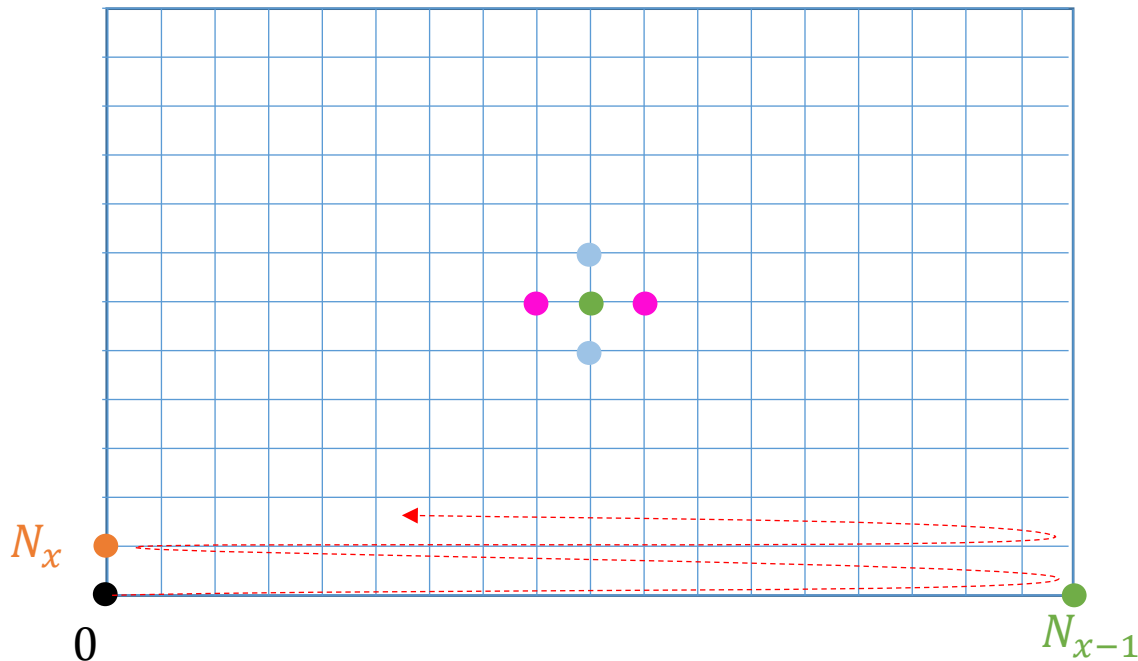


Voltage → Electric field
Current → Magnetic field

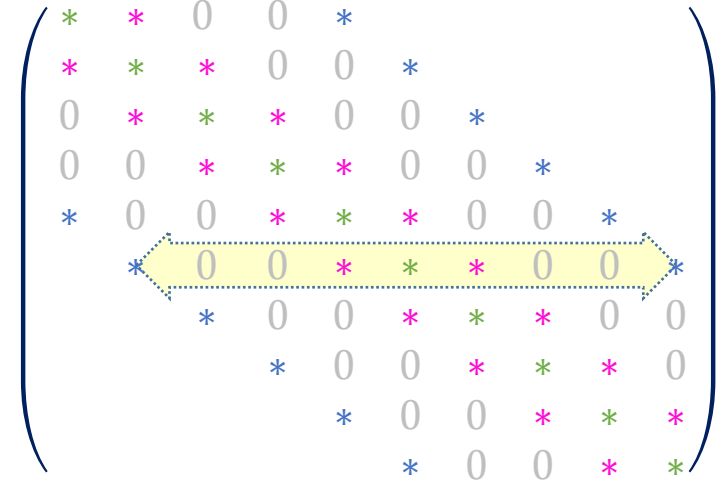
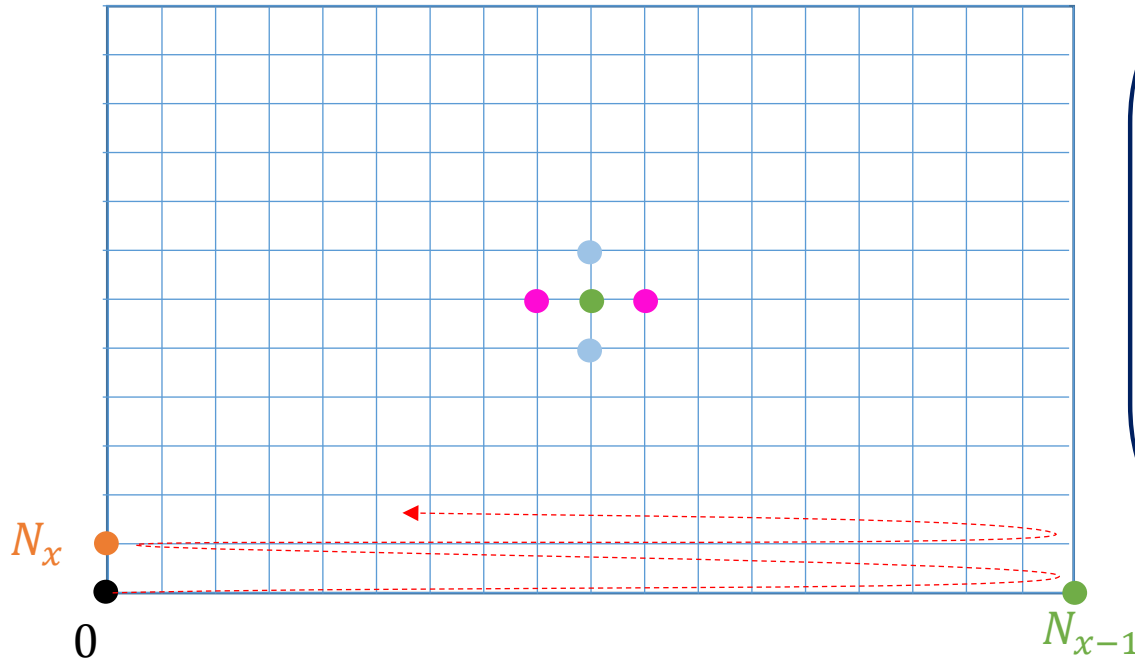
Same matrix pattern as for the 1D diffusion model

Example: What patterns do we observe in a matrix corresponding to 2D Diffusion and Wave Phenomena?

Solution: If we sample on a logically indexed 2D grid



What pattern does the nodal analysis matrix to solve this circuit give?



Note the samples surrounding each point indexed i are at

$$i - N_x, i - 1, i, i + 1, i + N_x$$

Example of a generally **banded matrix**

The efficiency of the specialised solving routines depends upon the bandwidth - yellow above - here $2N_x$

Example: Remember my concern in lecture 1 on nodal analysis matrices

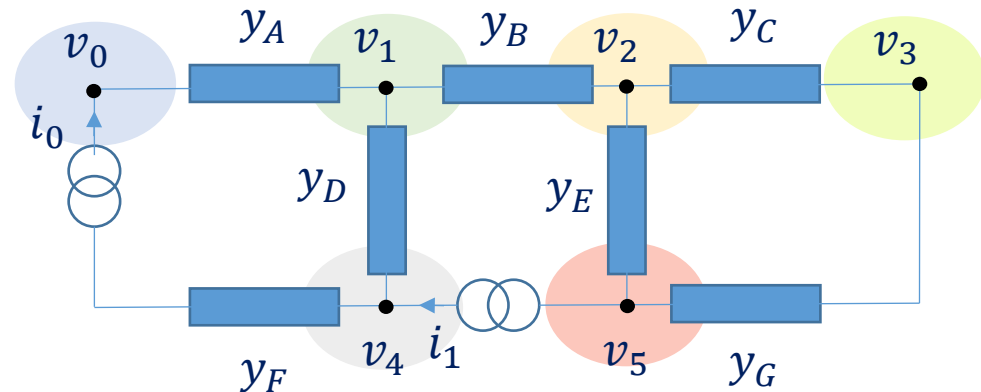
Did it matter in which order I wrote the nodal equations down in the matrix ?

Solution;

We now realise that the *result values* aren't affected, but using a nearest neighbour logic in the

unknown indexing gives a **quasi-banded**

matrix which means the matrix solving can be performed **far more efficiently**

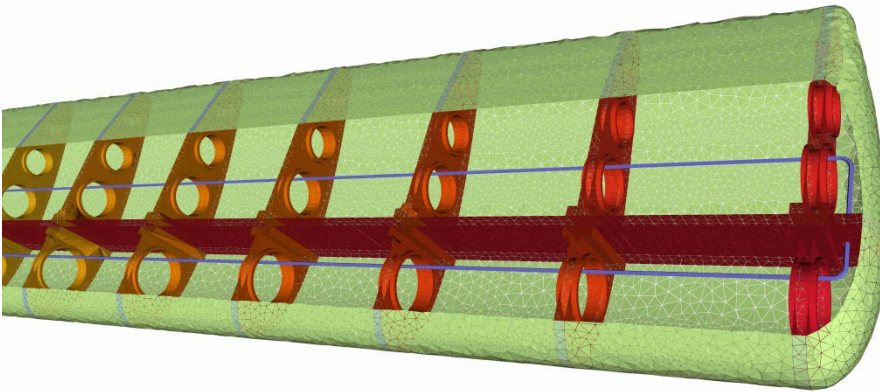


A subtlety: Actually for large problems, the excessive numerical evaluations in the presence of finite precision computing due to use of a *silly* indexing may actually also **undermine the accuracy** of the *result values*; so it's not just an issue of speed

Unstructured Mesh Modelling

Engineering modelling is a question of balancing issues

Example: We obtain *smoother* approximations of geometry by using a *triangular* description of surfaces than with the rectangular grid sampling of above

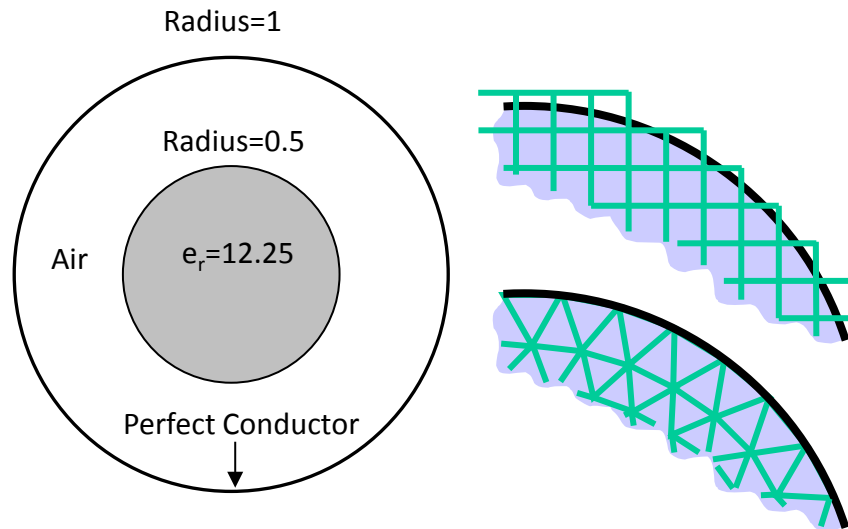


Nearest neighbour coupling: Each triangle has 3 neighbours, but cannot index "logically" to get a structured pattern. Special techniques determine best indexing for quasi-bounded

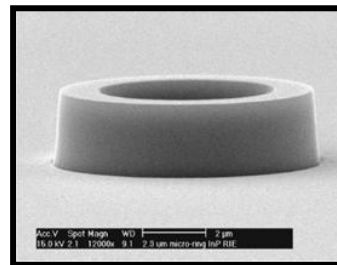
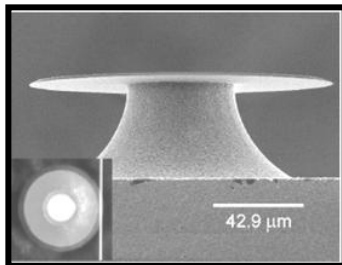
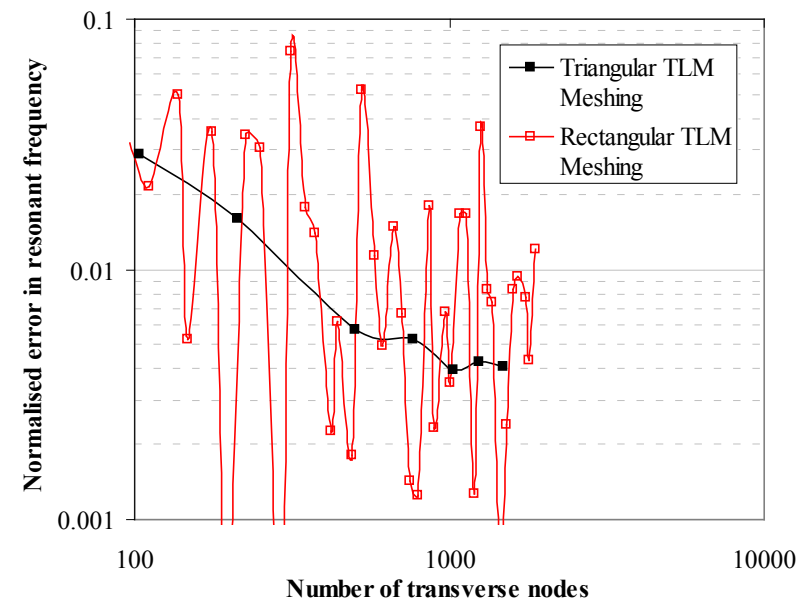
Pro: Fewer samples needed for a given quality of result; smaller matrices - faster?

Con: There is no *structure* to the sparse matrices; must use general iterative techniques

Example: Accuracy of resonant frequency identification of a circular dielectric loaded cavity depending on the number of samples used



E.g. to design practical photonic "filters"



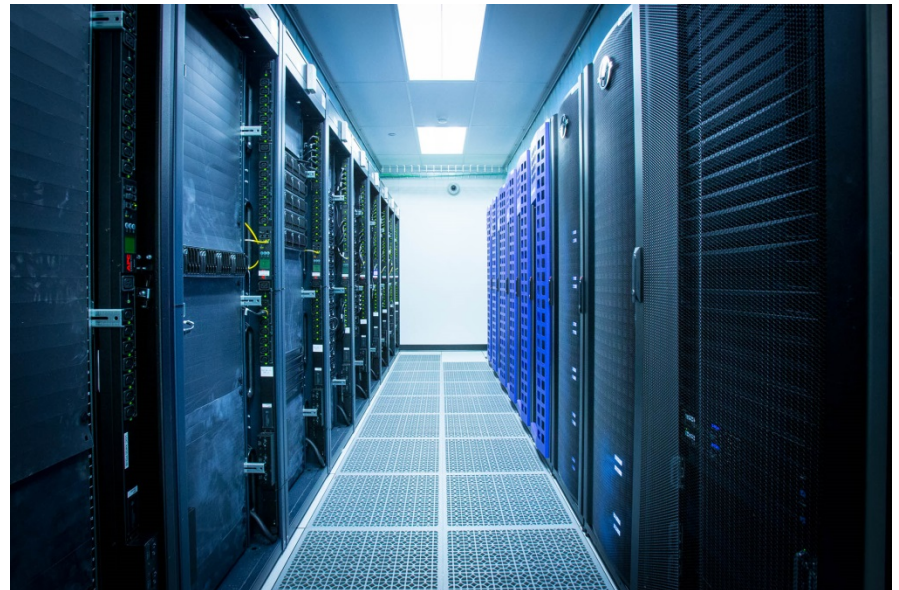
Can we take advantage of the Pattern of data in particular sparse matrices?

Absolutely and this is *good engineering*

We make choices and this introduces us to the whole field of *simulation algorithm design*

- Best sampling for best *capture* of geometry and *physics* with fewest samples - *matrix size*
- Dense or sparse matrices?
- Structured or unstructured matrices?
- "Admin": the order in which we index the samples has a massive impact on run time/memory!
- ... *and what about parallelisation?*

**If we have access to a 10,000 core 100TB
compute cluster - don't we just hit the
parallelise option to avoid long run times?**



If you believe that ...



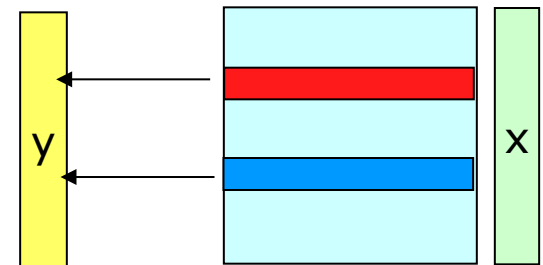
then I hope you like this other nice *fantasy land*

Parallelisation is a key enabler in the modern world

- Am I including this material as computing techniques are also a personal interest of mine? Not at all - the availability of parallel computing is a massive influence on the way in which realistic matrices are studied and processed
- However, the hardware of multicore CPUs is dictated by *commercial* needs so not ideal for number crunching. The limitation is often data throughput rate not just FLOPs, i.e. getting data in/out of memory

Matrix multiplication is easy to parallelise

Each element of the result, $\mathbf{y}$, could be evaluated by a separate CPU multiplying a row by $\mathbf{x}$



Non-iterative (i.e. direct) matrix solving is not so easy to parallelise

Example: LU decomposition algorithm for solving $Ax = b$

We have previously declared that all (square) matrices can be factorised as $A = LU$, e.g. for a 4 by 4 case, as

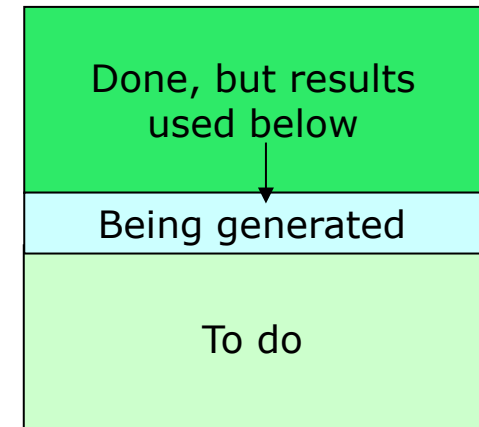
$$\begin{pmatrix} 1 & & & \\ L_{10} & 1 & & \\ L_{20} & L_{21} & 1 & \\ L_{30} & L_{31} & L_{32} & 1 \end{pmatrix} \begin{pmatrix} U_{00} & U_{01} & U_{02} & U_{03} \\ & U_{11} & U_{12} & U_{13} \\ & & U_{22} & U_{23} \\ & & & U_{33} \end{pmatrix}$$

Note that it is generally true that all the L_{ii} elements can be chosen to be 1

Practical algorithms return this packed as one matrix

$$\begin{pmatrix} U_{00} & U_{01} & U_{02} & U_{03} \\ L_{10} & U_{11} & U_{12} & U_{13} \\ L_{20} & L_{21} & U_{22} & U_{23} \\ L_{30} & L_{31} & L_{32} & U_{33} \end{pmatrix}$$

which is generated in a
sequential row at a
time manner



We cannot use a separate CPU to generate each row due to the need to know the result rows above - so does not obviously parallelise

Different views on parallelisation?

- **Either**, take advantage of multiple CPUs etc to *speed* up the algorithm you are already using - **low level parallelisation**; *illustrated by the matrix multiplication example above*

Low level parallelisation of existing algorithms may simply not be possible; illustrated by the LU example above

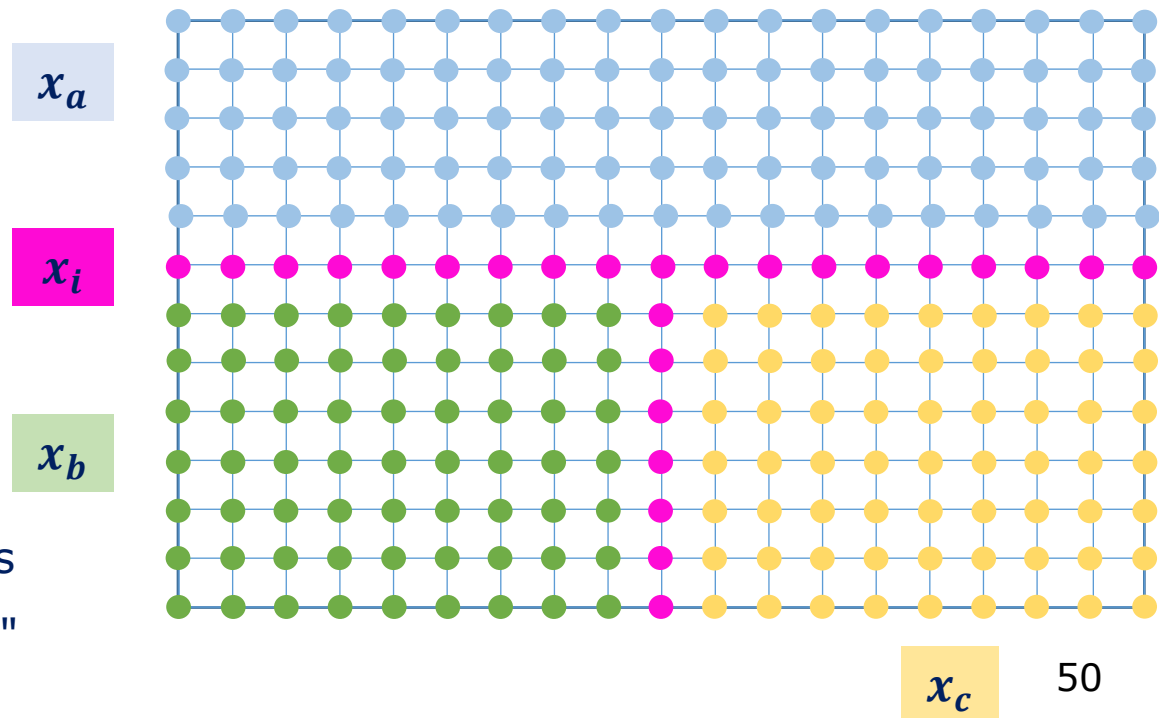
- **Or**, reconsider the overall problem and see if you can approach it in a different, *fundamentally parallel*, way - **high level parallelisation**; *illustrated by the next example*

A valuable class of parallelisation approaches are called **Domain decomposition methods** by engineers - also referred to as **block methods** in the maths community

In principle: Divide the problem into parts, allocate a CPU core to each part, then stitch together the parts

Example: Consider a sampled thermal model divided into 3 *domains* for solving $Ax = b$

- The unknowns are labelled as shown
- Note that the *interface* unknowns are identified
- Note no direct connections between the "big 3 blocks"



As the matrix is of the form "neighbour only coupling"

$Ax = b$ can be expanded to

$$\begin{pmatrix} \boxed{A_{aa} & & } & \boxed{A_{ai}} \\ & \boxed{A_{bb} & } & \boxed{A_{bi}} \\ & & \boxed{A_{cc}} & \boxed{A_{ci}} \\ \boxed{A_{ia}} & \boxed{A_{ib}} & \boxed{A_{ic}} & \boxed{A_{ii}} \end{pmatrix} \begin{pmatrix} x_a \\ x_b \\ x_c \\ x_i \end{pmatrix} = \begin{pmatrix} b_a \\ b_b \\ b_c \\ b_i \end{pmatrix}$$

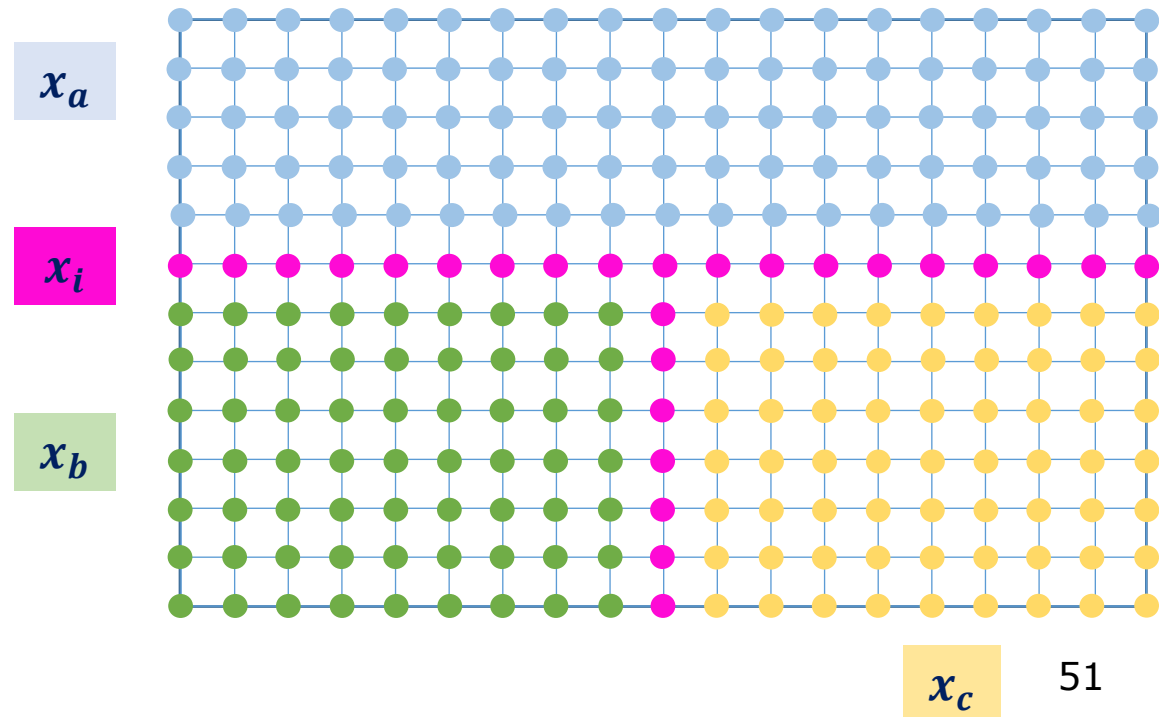
Notice how we have engineered a *block sparse* pattern

Besides general iterative methods applied to the whole matrix, we might consider

re-repressing this as

$$\begin{pmatrix} \tilde{A} & \tilde{B} \\ \tilde{C} & \tilde{D} \end{pmatrix} \begin{pmatrix} x_{abc} \\ x_i \end{pmatrix} = \begin{pmatrix} b_{abc} \\ b_i \end{pmatrix}$$

in order to clarify the essential types of blocks present



$$\begin{pmatrix} \boxed{A_{aa} & & } & \boxed{A_{ai} \\ & \boxed{A_{bb}} & & \boxed{A_{bi} \\ & & \boxed{A_{cc}} & \boxed{A_{ci} \\ \boxed{A_{ia}} & \boxed{A_{ib}} & \boxed{A_{ic}} & \boxed{A_{ii}} \end{pmatrix} \begin{pmatrix} x_a \\ x_b \\ x_c \\ x_i \end{pmatrix} = \begin{pmatrix} b_a \\ b_b \\ b_c \\ b_i \end{pmatrix} \rightarrow \begin{pmatrix} \tilde{A} & \tilde{B} \\ \tilde{C} & \tilde{D} \end{pmatrix} \begin{pmatrix} x_{abc} \\ x_i \end{pmatrix} = \begin{pmatrix} b_{abc} \\ b_i \end{pmatrix}$$

Notice that $\tilde{A}$ is a banded matrix so amenable to an efficiently evaluated LU decomposition $\tilde{A} = LU$ A few moments thought will confirm that

$$\begin{pmatrix} \tilde{A} & \tilde{B} \\ \tilde{C} & \tilde{D} \end{pmatrix} = \begin{pmatrix} L & 0 \\ \tilde{C}U^{-1} & I \end{pmatrix} \begin{pmatrix} U & L^{-1}\tilde{B} \\ 0 & \tilde{D} - \tilde{C}\tilde{A}^{-1}\tilde{B} \end{pmatrix}$$

Which is a *block LU decomposition for the whole matrix*

Why is this helpful?

Because we can use the standard LU solve approach and the only *unpleasant* (i.e. dense) matrix block is $\tilde{D} - \tilde{C}\tilde{A}^{-1}\tilde{B}$ which is *small* and *square*

$$\begin{pmatrix} \tilde{\mathbf{A}} & \tilde{\mathbf{B}} \\ \tilde{\mathbf{C}} & \tilde{\mathbf{D}} \end{pmatrix} \begin{pmatrix} \mathbf{x}_{abc} \\ \mathbf{x}_i \end{pmatrix} = \begin{pmatrix} \mathbf{L} & 0 \\ \tilde{\mathbf{C}}\mathbf{U}^{-1} & \mathbf{I} \end{pmatrix} \begin{pmatrix} \mathbf{U} & \mathbf{L}^{-1}\tilde{\mathbf{B}} \\ 0 & \tilde{\mathbf{D}} - \tilde{\mathbf{C}}\tilde{\mathbf{A}}^{-1}\tilde{\mathbf{B}} \end{pmatrix} \begin{pmatrix} \mathbf{x}_{abc} \\ \mathbf{x}_i \end{pmatrix} = \begin{pmatrix} \mathbf{b}_{abc} \\ \mathbf{b}_i \end{pmatrix}$$

As usual with LU, introduce the intermediate unknowns $\begin{pmatrix} \mathbf{L} & 0 \\ \tilde{\mathbf{C}}\mathbf{U}^{-1} & \mathbf{I} \end{pmatrix} \begin{pmatrix} \mathbf{y}_{abc} \\ \mathbf{y}_i \end{pmatrix} = \begin{pmatrix} \mathbf{b}_{abc} \\ \mathbf{b}_i \end{pmatrix}$

$$\mathbf{L} \mathbf{y}_{abc} = \mathbf{b}_{abc}$$

Simple Lower triangular solve

$$\mathbf{y}_i = \mathbf{b}_i - \tilde{\mathbf{C}}\mathbf{U}^{-1}\mathbf{y}_{abc}$$

Simple Upper triangular solve for $\mathbf{U}\mathbf{z} = \mathbf{y}_{abc}$ followed by

$$\mathbf{y}_i = \mathbf{b}_i - \tilde{\mathbf{C}}\mathbf{z}$$

Now solve the *upper block* problem

$$\begin{pmatrix} \mathbf{U} & \mathbf{L}^{-1}\tilde{\mathbf{B}} \\ 0 & \tilde{\mathbf{D}} - \tilde{\mathbf{C}}\tilde{\mathbf{A}}^{-1}\tilde{\mathbf{B}} \end{pmatrix} \begin{pmatrix} \mathbf{x}_{abc} \\ \mathbf{x}_i \end{pmatrix} = \begin{pmatrix} \mathbf{y}_{abc} \\ \mathbf{y}_i \end{pmatrix}$$

$$(\tilde{\mathbf{D}} - \tilde{\mathbf{C}}\tilde{\mathbf{A}}^{-1}\tilde{\mathbf{B}}) \mathbf{x}_i = \mathbf{y}_i$$

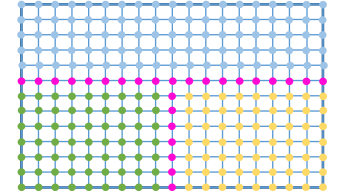
Dense matrix solve!

$$\mathbf{U}\mathbf{x}_{abc} = \mathbf{y}_{abc} - \mathbf{L}^{-1}\tilde{\mathbf{B}}\mathbf{x}_i$$

Simple Lower triangular solve for $\mathbf{L}\mathbf{w} = \tilde{\mathbf{B}}\mathbf{x}_i$ followed by
simple Upper triangular solver for $\mathbf{U}\mathbf{x}_{abc} = \mathbf{y}_{abc} - \mathbf{w}$

Questions: 1) We have a dense matrix solve! 2) How to "invert" to get $\tilde{A}^{-1}$
3) Where is the parallelisation?

1) The size of the dense solve, $(\tilde{D} - \tilde{C}\tilde{A}^{-1}\tilde{B}) x_i = y_i$ is that of the interface nodes - much smaller than that of the whole problem^a



2) The factorisation $\begin{pmatrix} \tilde{A} & \tilde{B} \\ \tilde{C} & \tilde{D} \end{pmatrix} = \begin{pmatrix} L & 0 \\ \tilde{C}U^{-1} & I \end{pmatrix} \begin{pmatrix} U & L^{-1}\tilde{B} \\ 0 & \tilde{D} - \tilde{C}\tilde{A}^{-1}\tilde{B} \end{pmatrix}$ can actually be done by an algorithm that **directly** gives the **actual elements** on the right hand side^b

3) The scope for parallelisation comes in the set of Upper and Lower solves involved. Note that $\tilde{A} = LU$ is sub-structured as

$$\tilde{A} = \begin{pmatrix} L_{aa}U_{aa} & & \\ & L_{bb}U_{bb} & \\ & & L_{cc}U_{cc} \end{pmatrix} = \begin{pmatrix} L_{aa} & & \\ & L_{bb} & \\ & & L_{cc} \end{pmatrix} \begin{pmatrix} U_{aa} & & \\ & U_{bb} & \\ & & U_{cc} \end{pmatrix}$$

So each "CPU" independently processes a block in the Upper and Lower solves

a) Note that $S = \tilde{D} - \tilde{C}\tilde{A}^{-1}\tilde{B}$, is called the Shur Complement matrix

b) See *Saad* Ref. in the Self Study Sheet

If we have access to a 10,000 core 100TB compute cluster - don't we just hit the *parallelise option* to avoid long run times?

- No such *easy use* option - major code rewrite required and typically a complete change of algorithm
- (Sparse) matrix multiplication and hence iterative techniques parallelise well
- Domain decomposition approaches are a powerful option